BUYWISE — AI/ML TECHNICAL SPECIFICATION & VIVA COMPENDIUM

A Deep-Dive into Algorithmic Decision-Making, Deterministic Capacity Formulations & Resilient Fallbacks

Course: B.Tech Computer Science & Engineering (AI/ML)	**Domain:** AI for Personal Finance
Engine Stack: FastAPI, Scikit-learn, NumPy, Pandas	**Target Audience:** Technical Reviewers & Viva Board
Author / Candidate: Project BuyWise Team	**Status:** Verified & Passing (53/53 Assertions)

1. Architectural Philosophy: The Hybrid Deterministic-ML Paradigm

A critical engineering requirement established for BuyWise is that **an AI or Machine Learning model must never act as the sole, opaque decision-maker over whether a student can afford a purchase.** Deep learning or statistical neural models can hallucinate, experience non-deterministic stochastic variations, and lack legal explainability.

BuyWise strictly bifurcates intelligence into two complementary layers:

- **Machine Learning Layer (Pattern Recognition & Discovery):** Identifies semantic categories from messy UPI transaction texts (TF-IDF + Logistic Regression), flags unusual expenditure deviations (Isolation Forest), and projects 30-day velocity trajectories (Linear Regression).
- **Deterministic Financial Decision Engine (Guaranteed Invariance):** Executes exact mathematical proofs against liquid balance, emergency reserves, remaining savings targets, and pending fee obligations. It is 100% explainable, deterministic, and transparent.

2. Natural Language Expense Categorization (TF-IDF + Logistic Regression)

2.1 What the Algorithm Does

TF-IDF (Term Frequency - Inverse Document Frequency) converts raw text strings into numerical sparse feature vectors. Multinomial Logistic Regression then models the probability distribution across 9 target expense classes (Food, Transport, Shopping, Education, Entertainment, Bills, Groceries, Healthcare, Savings) using a linear decision boundary squashed by the softmax activation function.

2.2 Mathematical Formulation

For a term t in document d within corpus D:

$$\textbf{TF}(t, d) = \text{count}(t \text{ in } d) / \text{total_words}(d)$$

$$\textbf{IDF}(t, D) = \ln((1 + |D|) / (1 + \text{df}(t))) + 1$$

$$\textbf{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D)$$

The softmax probability for class k given feature vector $\mathbf{x}$ is:

$$\textbf{P}(y = k \mid \mathbf{x}) = \exp(w_k^T x + b_k) / \sum_{j=1}^{K} \exp(w_j^T x + b_j)$$

2.3 Why it was Selected for BuyWise

Campus transactions feature concise 2-to-6 word strings ('Zomato lunch meal', 'Uber to railway station', 'Semester xerox printouts'). Complex Large Language Models (LLMs) or Transformer models (e.g. BERT) introduce latency (>1,200ms), large GPU RAM footprints, and recurrent API subscription costs. TF-IDF combined with Logistic Regression evaluates in under **2.4 milliseconds** with sub-megabyte RAM overhead and 100% on-device local execution.

2.4 Concrete Implementation & Example

Input: 'Amazon wireless noise cancelling headphones'
N-Gram Extraction: ['amazon', 'wireless', 'noise', 'cancelling', 'headphones', 'wireless noise', 'noise cancelling']
Softmax Output: Food: 0.02, Transport: 0.01, Shopping: **0.625**, Education: 0.12, Entertainment: 0.15
Result: Category: Shopping, Confidence: 0.625.

2.5 Advantages & Limitations

- *Advantages:* Ultra-fast inference (<3ms), highly interpretable feature weights (we can inspect which token drove the prediction), robust to unseen campus merchant variants.
- *Limitations:* Lacks deep syntactic context (bag-of-words assumption ignores word order beyond n-grams); novel regional slang not in vocabulary relies on character n-grams or deterministic keyword fallbacks.

3. Unsupervised Spending Outlier Detection (Isolation Forest)

3.1 What the Algorithm Does

Isolation Forest is an unsupervised tree-based ensemble algorithm that isolates anomalies instead of profiling normal data points. Anomalous transactions (such as a sudden ■4,500 coat purchase in a student account where average spend is ■150) have few neighbors and require significantly fewer recursive random feature splits to isolate in an isolation tree (iTree).

3.2 Mathematical Formulation

The anomaly score $s(x, n)$ for an instance x across an ensemble of n trees is:

$$\mathbf{s(x, n) = 2^{- (E(h(x)) / c(n))}}$$

where $h(x)$ is path length (number of edges traversed from root to termination), $E(h(x))$ is average path length across all iTrees, and $c(n) = 2(\ln(n-1) + 0.5772156649) - (2(n-1)/n)$ is the average path length of an unsuccessful binary search tree.
If $E(h(x)) \to 0$, score $\to 1$ (Outlier). If $E(h(x)) \to c(n)$, score $\to 0.5$ (Normal).

3.3 Why it was Selected for BuyWise

Students do not label their own historical spending as 'anomalous' or 'normal' (no ground-truth supervision). Unsupervised density estimators like GMMs or k-Means fail because student transactions exhibit multimodal clusters with high variance. Isolation Forest is $O(n \log n)$, invariant to linear scaling, and detects outliers without requiring prior distribution assumptions.

3.4 How BuyWise Uses It

The feature matrix inputs each transaction's normalized amount and category encoding. Outliers with *score > 0.58* or *prediction = -1* are flagged in Module 5 (Spending Intelligence) with human-readable notifications: *'Unusually high spend: ■4,500 for Shopping (mean: ■280, +3.8σ)'*.

3.5 Advantages & Limitations

- *Advantages:* No training labels required, linear time complexity, robust against masking effects (multiple close outliers).
- *Limitations:* Requires at least 5 to 10 transactions to construct stable isolation trees. When transactions < 5, BuyWise automatically activates a statistical Z-Score category fallback.

4. Short-Horizon Expenditure Forecasting (Ordinary Least Squares Linear Regression)

4.1 What the Algorithm Does

Ordinary Least Squares (OLS) Linear Regression fits a straight line through chronological daily spending totals by minimizing the sum of squared differences (residuals) between observed daily expenses and the fitted model. It models the spending trajectory slope β_1 to detect accelerating vs. decelerating burn rates.

4.2 Mathematical Formulation

$$y■ = \beta_0 + \beta_1 \times t$$
$$\beta_1 = \Sigma((t_i - t■)(y_i - y■)) / \Sigma(t_i - t■)^2$$
$$\beta_0 = y■ - \beta_1 \times t■$$

```
Trend Classification:
    If β1 > 2.5: accelerating_spending (Burn rate increasing daily)
    If β1 < -2.5: decelerating_spending (Spending tapering off)
    Else: stable_spending
```

4.3 Why it was Selected for BuyWise

Students evaluate finances over monthly allowance horizons (30 days). Complex ARIMA or LSTM recurrent networks require hundreds of continuous daily samples with stationary seasonality, which college students rarely have. Linear regression with moving-average bounds yields robust, noise-tolerant 30-day forward projections without overfitting on sporadic spikes.

5. The Deterministic Financial Decision Engine & Mathematical Formulations

5.1 Core Formulas

The core mathematical framework enforces absolute consistency across all modules:

1. **Remaining Savings Target (RST):**
   RST = max(0, Monthly_Savings_Target - Actual_Savings)

2. **Total Protected Reserves (TPR):**
   TPR = Emergency_Reserve + RST

3. **Safe Discretionary Spending Capacity (SDC):**
   SDC = max(0, Current_Liquid_Balance - TPR)

4. **Projected Remaining Balance (PRB):**
   PRB = Current_Liquid_Balance - Purchase_Amount

5. **Buffer After Obligations (BAO):**
   BAO = Current_Liquid_Balance - Purchase_Amount - Upcoming_Commitments_Total

5.2 The 5 Mutually-Exclusive Decision Branches

Branch Condition	Recommendation	Governing Logic
PRB < 0	**RECONSIDER** (or **WAIT** if income restores)	Purchase price exceeds entire liquid cash balance.
PRB < Emergency_Reserve	**RECONSIDER** (or **WAIT** if income restores)	Violates untouched emergency reserve buffer.
PRB < TPR (Protected Reserves)	**RECONSIDER** (or **WAIT** if income restores)	Preserves emergency reserve, but encroaches on remaining savings target.
BAO < TPR (Due Bills)	**RECONSIDER** (or **WAIT** if income restores)	Jeopardizes upcoming college fees or rent due within 30 days.
PRB ≥ TPR & BAO ≥ TPR	**BUY NOW**	Comfortably affordable above all reserves and commitments.

6. Fault-Tolerant Statistical Fallbacks & Graceful Degradation

A foundational tenet of production-grade software engineering is **graceful degradation**. If the Python FastAPI ML process is terminated, undergoing retraining, or network-isolated, BuyWise never crashes or throws 500 errors. Instead, the Node.js backend immediately switches to calibrated deterministic and statistical fallbacks:

ML Capability	Primary ML Model	Fallback Architecture
Expense Categorization	TF-IDF + Multinomial Logistic Regression	Keyword Dictionary Matching across 9 categories with normalized substring scoring (e.g. 'swiggy'→Food, 'uber'→Transport).
Anomaly Detection	Scikit-learn Isolation Forest Ensemble	Category Z-Score Deviation (flags transactions with z > 2.0σ and amount > ■500 against category mean).
Spending Forecasting	OLS Linear Regression Trajectory	Time-Weighted Daily Moving Average over transaction span days.

7. Academic Viva Defense Cheat Sheet (Top 7 Critical Questions)

Q1: Why did you not use an LLM (like GPT-4) as the purchase decision evaluator?

LLMs are generative stochastic models prone to hallucination, latency (>1.5s), and non-determinism. Financial gating requires strict mathematical invariance: a ■1,800 purchase must consistently yield the exact same verifiable recommendation every time. LLMs are appropriate for conversational summaries, but financial decision engines must remain deterministic.

Q2: How does the system prevent double-counting when money is transferred to savings?

When ■400 is transferred to savings, the liquid spending balance decreases from ■3,500 to ■3,100, while actualSavings increments from ■0 to ■400. This dynamically reduces the remainingSavingsTarget from ■1,000 to ■600. Because TPR = 700 + 600 = 1,300, safe discretionary spending remains invariant at 3,100 - 1,300 = ■1,800.

Q3: What happens when a new monthly savings cycle begins?

BuyWise executes `checkAndRolloverSavingsCycle`. The active cycle's actualSavings resets to ■0 so the student is challenged to fulfill the new month's ■1,000 target. Meanwhile, totalAccumulatedSavings carries forward the lifetime wealth (■400), ensuring savings are never returned to liquid spendable cash.

Q4: Why use Isolation Forest instead of K-Means for anomaly detection?

K-Means is a clustering algorithm, not an outlier detector; it forcibly assigns every point (including severe anomalies) to a cluster center. Furthermore, student transactions exhibit multimodal clusters with high variance. Isolation Forest explicitly isolates outliers using random partitioning trees in O(n log n) time without spherical cluster assumptions.

Q5: How does your categorizer handle imbalanced campus spending data?

During training on the 114 campus dataset, Logistic Regression was configured with `class_weight='balanced'`. This penalizes misclassifications in low-frequency classes (like Healthcare or Bills) inversely proportional to their class frequency, preventing majority classes (Food, Shopping) from dominating.

Q6: What is the computational complexity of your decision engine?

The decision engine evaluates in O(k) time where k is the number of pending obligations in the 30-day evaluation horizon (typically k < 10). It executes in under 1 millisecond on any commodity CPU without GPU acceleration.

Q7: How is user data isolated in the multi-tenant MongoDB architecture?

Every database query across profiles, transactions, obligations, and purchase decisions is strictly scoped with `{ userId: req.user.id }` extracted from the cryptographically verified JWT token. User B can never read or mutate User A's financial assets.